DogApp

Agenda de citas para mascotas

Universidad del Valle

Escuela de Ingeniería de Sistemas y Computación · Sede Cali

Documento de construcción

Miniproyecto 1 — Sprint 1

Qué hay que construir, cómo, y cómo se gana cada décima de la rúbrica de evaluación

Curso	Desarrollo de Aplicaciones para Dispositivos Móviles
Docente	Ing. Walter Medina
Proyecto	DogApp — Agenda de citas para mascotas (5 Historias de Usuario)
Alcance	Sprint 1: HU 1.0 a HU 5.0 + rúbrica RA-1, RA-2, RA-3
Sprint	11 de noviembre → 25 de noviembre de 2026, 6:30 p.m.
Monitoría	18 de noviembre de 2026
Entregable	PDF `equipoUno.pdf` (o el nombre del equipo) + repositorio + Jira

Documento de trabajo del equipo. Elaborado a partir del enunciado oficial de las Historias de Usuario y de la tabla de evaluación del Miniproyecto 1.

Índice

1. Qué se construye, en una página **3**
1.1. Las cinco pantallas . . . 3
1.2. Las siete capacidades técnicas que exige la rúbrica . . . 3

2. La rúbrica manda: cómo se reparte la nota **4**
2.1. La nota individual multiplica (no promedia) . . . 4
2.2. Penalizaciones explícitas . . . 5
2.3. Mapa: de la rúbrica al artefacto concreto . . . 5

3. Arquitectura obligatoria: MVVM + Repository **5**
3.1. El flujo que hay que poder dibujar en el tablero . . . 6
3.2. Reglas de oro que el jurado verifica . . . 6
3.3. Paquetes y clases que se deben crear . . . 7

4. Modelo de datos y persistencia con Room **7**
4.1. La entidad . . . 8
4.2. DAO: las cinco operaciones del sprint . . . 8
4.3. Base de datos y Repository . . . 9

5. Consumo de API REST con Retrofit **10**
5.1. Configuración de Retrofit . . . 11
5.2. Reglas de consumo que evitan perder puntos . . . 11

6. Sistema de diseño extraído de los pantallazos **12**
6.1. Tokens de color . . . 12
6.2. Formas, elevaciones y tipografía . . . 12
6.3. Fuentes . . . 13
6.4. Archivos de recursos a crear . . . 13

7. Especificación pantalla por pantalla **13**
7.1. HU 1.0 — Ventana Login . . . 13
7.2. HU 2.0 — Home Administrador de Citas . . . 15
7.3. HU 3.0 — Ventana Nueva Cita . . . 16
7.4. HU 4.0 — Ventana Detalle de la Cita . . . 17
7.5. HU 5.0 — Ventana Editar Cita . . . 19

8. Navegación: un solo Activity, cinco Fragments **19**

9. Jira y GitHub: el 20 % que suele perderse por descuido **20**

9.1. Jira: cómo reflejar los criterios 20

9.2. GitHub: reglas del repositorio 21

10. Plan de ejecución del sprint **22**

11. Errores que cuestan nota (y su antídoto) **23**

12. Checklist de aceptación **23**

12.1. Arquitectura (RA-1, 60 %) 24

12.2. Interfaz (RA-2, 20 %) 24

12.3. Funcionamiento y gestión (RA-1 funcional + RA-3) 25

13. Entrega y sustentación **25**

13.1. Guion sugerido de sustentación (10–12 minutos) 25

14. Anexo A: dependencias de `build.gradle.kts` **27**

15. Anexo B: código mínimo de las tres piezas delicadas **28**

1 Qué se construye, en una página

DogApp es una aplicación Android nativa para que el administrador de una veterinaria registre y gestione las citas de las mascotas que debe atender. El Sprint 1 entrega **cinco pantallas** y **siete capacidades técnicas obligatorias**.

1.1 Las cinco pantallas

1. **Login (HU 1.0):** pantalla de bienvenida oscura con la marca DogApp y una huella animada; al tocarla se abre la autenticación biométrica del dispositivo y, si el dedo es reconocido, se entra a la aplicación.
2. **Home Administrador de Citas (HU 2.0):** lista de citas guardadas en una base de datos local, cada una con foto circular de la mascota, nombre, síntoma y turno; botón flotante rosado para crear una cita.
3. **Nueva Cita (HU 3.0):** formulario con cuatro campos validados y una lista de síntomas; el botón Guardar solo se habilita cuando todo está completo.
4. **Detalle de la Cita (HU 4.0):** ficha del paciente con dos tarjetas superpuestas y dos botones flotantes: eliminar (rosado) y editar (azul oscuro).
5. **Editar Cita (HU 5.0):** el mismo formulario, precargado, que actualiza el registro y regenera la foto según la nueva raza.

1.2 Las siete capacidades técnicas que exige la rúbrica

La tabla de evaluación del enunciado lista exactamente estos siete items dentro del RA-1 (60 %). Si falta uno, se pierde la fracción correspondiente de ese 60 %:

#	Capacidad	Dónde se evidencia
1	Arquitectura MVVM **con Repository**	Paquetes `model`, `view`, `viewmodel`, `repository`, `data`
2	Funcionamiento según las Historias de Usuario	Las 5 pantallas y sus criterios
3	Base de datos SQLite con **Room**	Entidad, DAO, Database, Repository
4	Consumo de API REST	Retrofit contra `dog.ceo`
5	Uso de **Fragments**	5 fragments, no 5 Activities
6	Uso de **Navigation**	`nav_graph.xml` con acciones y argumentos
7	Interfaz idéntica a los pantallazos (RA-2)	Sistema de diseño de la sección 6

Fuera de alcance del Sprint 1 — no lo construyas

No hay autenticación con usuario y contraseña, ni registro, ni roles, ni recuperación de clave. No hay notificaciones, ni calendario, ni recordatorios, ni historial clínico, ni sincronización en la nube. No hay pantalla de configuración. La autenticación es *exclusivamente* biométrica dactilar, y la persistencia es *exclusivamente* local con Room. Invertir tiempo aquí no suma nota y resta tiempo al sprint.

2 La rúbrica manda: cómo se reparte la nota

La nota del equipo **no depende de que la app se vea bonita**: depende de que cada item de la tabla se pueda demostrar en la sustentación. Esta es la distribución textual del enunciado.

RA	%	Items a calificar
RA-1. Implementa aplicaciones nativas para una plataforma móvil	**60**	Uso de arquitectura MVVM (con Repository) · Funcionamiento de la app como se indica en las HU · Uso de base de datos SQLite con Room · Consumo de API REST · Uso de Fragment · Uso de Navigation
RA-2. Usa principios de Diseño de Experiencia y de Interacción	**20**	Interfaz de la app como se indica en las Historias de Usuario
RA-3. Emplea gestión de proyectos y control de versión	**20**	Gestión de las tareas (criterios) de las HU en Jira: "Por hacer, en curso, hecho" · Uso de GitHub donde se evidencie activamente el trabajo en equipo durante el sprint
Nota equipo	**100 %**	

2.1 La nota individual multiplica (no promedia)

Nota definitiva del estudiante = Nota equipo × Nota individual

Ejemplo del enunciado: equipo 5 × individual 0,8 = **4,0**

La nota individual va de 0 a 1 y el docente la obtiene de *"corroborar que cada uno de los integrantes participó activamente"*. En la práctica eso se lee en tres lugares, y por eso el RA-3 no es trámite:

- **Commits de GitHub con el nombre de cada integrante** (autoría real, no un solo compañero subiendo todo).
- **Movimiento de tarjetas en Jira** con responsable asignado.
- **Capacidad de cada uno de explicar y modificar su parte en vivo** durante la sustentación.

Un equipo con la app perfecta donde solo dos personas tienen commits puede terminar con notas individuales de 0,5 para el resto. **El reparto de tareas es parte del diseño técnico, no un adorno.**

2.2 Penalizaciones explícitas

Situación	Efecto
No encender la cámara durante la sustentación	−0,5 en la nota final
Commit después de la fecha y hora de entrega	−0,2 en la nota final
Cualquier indicio de copia	Sanciones de los estatutos de la Universidad
Presentar de forma individual (la presentación es en equipo)	No admitido

2.3 Mapa: de la rúbrica al artefacto concreto

Esta es la tabla que conviene tener abierta mientras se trabaja. Cada fila es un item calificable y el artefacto exacto que lo demuestra.

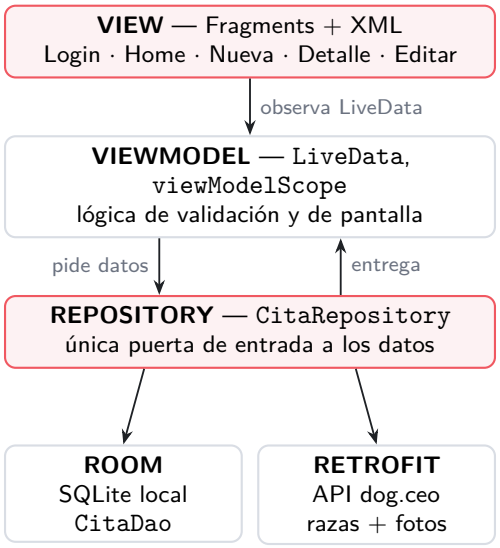
RA	Item	Artefacto que lo demuestra
RA-1	MVVM + Repository	Estructura de paquetes + `CitaViewModel` observando `CitaRepository`; ninguna consulta a Room ni a Retrofit dentro de un Fragment
RA-1	Funcionamiento HU	Recorrido completo: login → home → nueva → home → detalle → editar → home → detalle → borrar
RA-1	Room	`CitaEntity`, `CitaDao`, `DogAppDatabase` y base persistente tras cerrar la app
RA-1	API REST	`DogApiService` con Retrofit: lista de razas y foto por raza
RA-1	Fragment	`LoginFragment`, `HomeFragment`, `NuevaCitaFragment`, `DetalleFragment`, `EditarFragment`
RA-1	Navigation	`res/navigation/nav_graph.xml` con 5 destinos, 6–8 acciones y argumento `citaId`
RA-2	Interfaz	Comparación lado a lado con los 7 pantallazos del enunciado
RA-3	Jira	Tablero con columnas Por hacer / En curso / Hecho e historias moviéndose durante 2 semanas
RA-3	GitHub	Repositorio `equipoUno`, ramas por HU, Pull Requests y commits de todos

3 Arquitectura obligatoria: MVVM + Repository

Las diapositivas de la clase 9 fijan la definición que el docente va a usar como vara: *Model* (datos, sin conocer la interfaz), *View* (Activity o Fragment, solo lógica de interfaz), *ViewModel* (conecta Model con View por medio de LiveData y maneja la lógica) y *Repository* (puente entre las fuentes de datos y

el ViewModel). Sobre eso, la rúbrica agrega dos capacidades observables: Room para SQLite y Retrofit para la API.

3.1 El flujo que hay que poder dibujar en el tablero



3.2 Reglas de oro que el jurado verifica

1. **Ningún Fragment toca Room ni Retrofit.** Ni un `findViewById` de base de datos, ni un `Call` de Retrofit dentro de la vista. Todo pasa por el ViewModel y éste por el Repository.
2. **Ningún ViewModel conoce `Context`, `View` ni `Activity`.** Si aparece `Toast` dentro de un ViewModel, ya hay una fuga de capa.
3. **Toda la UI se actualiza observando `LiveData`** (o `StateFlow` si el equipo domina corrutinas; el enunciado de la clase habla de LiveData, así que LiveData es la apuesta segura).
4. **Las operaciones de base de datos son `suspend`** y se invocan con `viewModelScope.launch`.
5. **El Repository es la única fuente de verdad.**
6. **Una sola Activity** (`MainActivity`) con un `NavHostFragment`; las cinco pantallas son Fragments.

Señal de alarma para la nota de arquitectura

Si el profesor pide "muéstrame el ViewModel de Nueva Cita" y lo que aparece es un Fragment de 300 líneas con `db.citaDao().insert(...)`, el item MVVM vale cero aunque la app funcione. La app tiene que funcionar *y* estar partida en capas.

3.3 Paquetes y clases que se deben crear

```
com.univalle.dogapp
|-- data/
|   |-- local/
|   |   |-- CitaEntity.kt          @Entity(tableName = "citas")
|   |   |-- CitaDao.kt             @Dao
|   |   `-- DogAppDatabase.kt      @Database(entities=[CitaEntity::class], version=1)
|   `-- remote/
|       |-- DogApiService.kt       @GET breeds/list/all, breed/{raza}/images/random
|       |-- BreedsResponse.kt      data class (message: Map<String, List<String>>)
|       `-- ImageResponse.kt       data class (message: String, status: String)
|-- repository/
|   `-- CitaRepository.kt          une DAO + API + Glide
|-- model/
|   `-- Cita.kt                    modelo de dominio / UI
|-- viewmodel/
|   |-- LoginViewModel.kt          estado de autenticacion
|   |-- HomeViewModel.kt           LiveData<List<Cita>>
|   |-- FormCitaViewModel.kt       crea y edita + razas + foto
|   `-- DetalleViewModel.kt        LiveData<Cita?> + eliminar
|-- view/
|   |-- MainActivity.kt            host del NavHostFragment
|   |-- login/LoginFragment.kt
|   |-- home/HomeFragment.kt  +  CitaAdapter.kt  +  CitaViewHolder.kt
|   |-- nueva/NuevaCitaFragment.kt
|   |-- detalle/DetalleFragment.kt
|   `-- editar/EditarFragment.kt
`-- utils/
    |-- Constants.kt               BASE_URL, nombres de argumentos
    `-- BiometricHelper.kt         envoltorio de BiometricPrompt
```

Atajo legítimo: el proyecto de la clase 8

El proyecto `clase8_univalle` que acompaña el curso tiene *exactamente* la misma arquitectura que pide este miniproyecto, pero con un inventario: `InventoryDao`, `InventoryDB`, `InventoryRepository`, `InventoryViewModel`, cuatro Fragments (Home, Add, Details, Edit), `nav_graph.xml`, RecyclerView con adapter y viewholder, y Room 2.5.2 con KSP. Sirve como **esqueleto de referencia del docente**: renombrar a `Cita`, cambiar el modelo de datos, añadir Retrofit y la biometría, y rehacer por completo la interfaz. Usar material del curso no es copia; copiar el repositorio de otro equipo sí lo es.

4 Modelo de datos y persistencia con Room

4.1 La entidad

El enunciado pide cinco datos más el turno. El turno **es el ID autogenerado** de la base de datos, y la foto depende de la raza, así que hay que guardar también la URL de la imagen.

Campo	Tipo	Regla del enunciado	Nulo
`id`	`Int`	`@PrimaryKey(autoGenerate = true)`. Es el **turno** que se pinta como `#1`, `#2`… en el Home y en el Detalle	✗
`nombreMascota`	`String`	Obligatorio, máximo 15 caracteres	✗
`raza`	`String`	Obligatorio; se elige del autocompletado de razas	✗
`propietario`	`String`	Obligatorio, máximo 30 caracteres	✗
`telefono`	`String`	Obligatorio, solo dígitos, máximo 10	✗
`sintoma`	`String`	Uno de los seis síntomas de la lista	✗
`imagenUrl`	`String?`	URL que devuelve la API para esa raza	sí
`fechaCreacion`	`Long`	`System.currentTimeMillis()`; desempata el orden	✗

Dos decisiones que evitan bugs clásicos

(a) Guarda `telefono` como `String`, no como `Int`: si se guarda como entero, un número que empiece por 0 pierde el cero y un teléfono con espacios revienta el parseo. **(b)** Guarda la `raza` tal como se muestra (“Pug”, “Pitbull”), pero para llamar a la API normaliza en el Repository (`raza.trim().lowercase()`) porque el endpoint espera la clave en minúsculas. No guardes la clave de la API como texto visible del usuario.

4.2 DAO: las cinco operaciones del sprint

```
@Dao
interface CitaDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(cita: CitaEntity): Long      // devuelve el turno nuevo

    @Update
    suspend fun actualizar(cita: CitaEntity)

    @Delete
    suspend fun eliminar(cita: CitaEntity)

    @Query("SELECT * FROM citas ORDER BY id ASC")
    fun observarTodas(): LiveData<List<CitaEntity>>   // reactivo para el Home

    @Query("SELECT * FROM citas WHERE id = :id")
    fun observarPorId(id: Int): LiveData<CitaEntity?> // para Detalle y Editar
}
```

El detalle que rompe la mitad de los proyectos

Si `observarTodas()` devuelve `List<CitaEntity>` en lugar de `LiveData<...>`, la lista del Home **no se refresca** después de guardar o eliminar, y hay que llamar a mano al método cada vez. Con `LiveData`, la pantalla se actualiza sola y el criterio "debe visualizarse la cita guardada" (HU 3.0 y HU 5.0) se cumple sin código extra. Es también la evidencia de "LiveData" de la clase 9.

4.3 Base de datos y Repository

```
@Database(entities = [CitaEntity::class], version = 1, exportSchema = false)
abstract class DogAppDatabase : RoomDatabase() {
    abstract fun citaDao(): CitaDao
    companion object {
        @Volatile private var INSTANCIA: DogAppDatabase? = null
        fun obtener(context: Context): DogAppDatabase =
            INSTANCIA ?: synchronized(this) {
                INSTANCIA ?: Room.databaseBuilder(
                    context.applicationContext,
                    DogAppDatabase::class.java, "dogapp.db"
                ).build().also { INSTANCIA = it }
            }
    }
}
```

El `CitaRepository` recibe el DAO y el servicio de Retrofit y expone funciones de negocio, no de base de datos:

```
class CitaRepository(private val dao: CitaDao, private val api: DogApiService) {

    fun citas(): LiveData<List<CitaEntity>> = dao.observarTodas()
    fun cita(id: Int): LiveData<CitaEntity?> = dao.observarPorId(id)

    suspend fun guardar(cita: CitaEntity): Long = dao.insertar(cita)
    suspend fun modificar(cita: CitaEntity) = dao.actualizar(cita)
    suspend fun borrar(cita: CitaEntity) = dao.eliminar(cita)

    suspend fun razas(): List<String> = withContext(Dispatchers.IO) {
        api.listarRazas().message.keys
            .map { it.replaceFirstChar { c -> c.uppercase() } }
            .sorted()
    }

    suspend fun fotoDeRaza(raza: String): String? = withContext(Dispatchers.IO) {
        runCatching { api.fotoPorRaza(raza.trim().lowercase()).message }.getOrNull()
    }
}
```

Inyección de dependencias sin frameworks

No hace falta Hilt ni Koin. Basta una clase `ServiceLocator` (o un `object AppContainer`) que construya la base de datos, el servicio Retrofit y el repository una sola vez, y fábricas de ViewModel (`ViewModelProvider.Factory`) que los reciban. La rúbrica no pide DI; sí pide que las capas estén separadas.

5 Consumo de API REST con Retrofit

La app consume dos servicios públicos de `dog.ceo`; ambos son GET y no requieren llave.

Uso	Endpoint	Respuesta
Lista de razas para el autocompletado	`https://dog.ceo/api/breeds/list/all`	`message` es un objeto del tipo `{"raza": ["subrazas"]}`: `pug` viene con lista vacía y `bulldog` con `["boston", "english"]`
Foto de una raza concreta	`https://dog.ceo/api/breed/{raza}/images/random`	`message` trae la URL de una foto de esa raza y `status` vale `"success"`

☞ El enunciado sugiere `breeds/image/random`: leélo con cuidado

El texto de la HU 3.0 y HU 5.0 dice que para la foto se debe consumir `https://dog.ceo/api/breeds/image/random`, pero ese endpoint devuelve una **foto de una raza aleatoria**, no de la raza elegida. Y el mismo enunciado exige que "si la raza escogida fue Akita, la imagen debe mostrar un perro alusivo a esta raza". Las dos frases no se cumplen con el mismo endpoint. **Solución defendible:** usar `/breed/{raza}/images/random` para cumplir el requisito funcional, y dejar `breeds/image/random` como *fallback* cuando la raza no exista en la API (por ejemplo si el usuario escribe una raza a mano) o cuando falle la red. Menciónalo en el README y en la sustentación: demuestra criterio, no error.

5.1 Configuración de Retrofit

```
interface DogApiService {
    @GET("api/breeds/list/all")
    suspend fun listarRazas(): BreedsResponse

    @GET("api/breed/{raza}/images/random")
    suspend fun fotoPorRaza(@Path("raza") raza: String): ImageResponse

    @GET("api/breeds/image/random")
    suspend fun fotoAleatoria(): ImageResponse
}

object RetrofitClient {
    private const val BASE_URL = "https://dog.ceo/"
    val api: DogApiService by lazy {
        val log = HttpLoggingInterceptor().apply { level = BODY }
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .client(OkHttpClient.Builder().addInterceptor(log).build())
            .build()
            .create(DogApiService::class.java)
    }
}
```

5.2 Reglas de consumo que evitan perder puntos

1. **Una sola llamada a la lista de razas por sesión**, no una por tecla. Se carga en el `init` del ViewModel, se expone como `LiveData<List<String>>` y el adapter filtra en memoria.
2. **El filtro del autocompletado es `startsWith`**, no `contains`: el criterio dice "que comiencen por esas letras". Umbral: 2 caracteres (`android:completionThreshold="2"`).
3. **Todo en corrutinas**: `suspend` + `Dispatchers.IO`. Nada de red en el hilo principal (NetworkOnMainThreadException en 2 segundos).
4. **Tolera el fallo**: si no hay internet, la app tiene que seguir creando citas con una imagen de marcador. Envuelve con `runCatching` y no dejes que una excepción cierre la pantalla.
5. **Rotación**: un `Retrofit` por proceso, no uno por Fragment.
6. **Manifiesto**: `<uses-permission android:name="android.permission.INTERNET"/>`.
7. **Carga de imágenes con Glide** (o Picasso/Coil): `Glide.with(view).load(url)` con `placeholder(R.drawable.ic_dog_placeholder)` y `into(imageView)` evita bloqueos, memoriza y recorta al círculo con `circleCrop()`.

6 Sistema de diseño extraído de los pantallazos

El enunciado advierte que no hay prototipo con medidas exactas y que hay que parecerse "lo más posible" a los pantallazos. Estos son los valores que se leen de las capturas y que hay que centralizar en `res/values` para que las cinco pantallas queden consistentes (eso es el RA-2).

6.1 Tokens de color

Token	Valor	Uso
`dogapp_pink`	`#F05861`	Botones Guardar/Editar, FAB de crear, flechas de atrás, turno, línea divisoria del detalle, FAB de eliminar
`dogapp_dark`	`#2E2E33`	Fondo del login y de todos los headers/toolbars
`dogapp_pink_disabled`	`#F5A9AD`	Estado inactivo del botón Guardar Cita
`dogapp_field_focus`	`#6D3FCB`	Borde del campo cuando tiene el foco (morado)
`dogapp_edit_blue`	`#2B2B6B`	FAB de editar en el detalle
`white`	`#FFFFFF`	Texto sobre oscuro, fondo de tarjetas, fondo de formularios
`hint_gray`	`#9E9E9E`	Hints de los campos y textos secundarios
`ink`	`#212121`	Nombres, razas y contenido principal

6.2 Formas, elevaciones y tipografía

Elemento	Radio	Elevación	Nota
Header del Home	48 dp abajo-izquierda	4 dp	Solo la esquina inferior izquierda curvada; la derecha queda recta
Toolbar Nueva/Detalle/Editar	28 dp abajo (ambas)	4 dp	Esquinas inferiores curvadas, ver pantallazos
Tarjeta de ítem (Home)	16 dp	4 dp	Fondo blanco sobre fondo blanco: la separación la da la elevación y un margen de 8 dp
Tarjeta de imagen (Detalle)	16 dp	8 dp	Va *encima* de la tarjeta de datos
Tarjeta de datos (Detalle)	16 dp	4 dp	Se solapa por debajo de la de la imagen
Botón Guardar/Editar	24 dp (cápsula)	0–2 dp	Rosa, texto blanco bold cuando está habilitado
Campo de texto	8 dp	0	`OutlinedBox`, borde gris 1 dp y morado 2 dp con foco
FAB	16 dp	6 dp	56 dp; crear (abajo-der), borrar (abajo-izq), editar (abajo-der)
Avatar circular	50 %	0	52 dp, `circleCrop`

6.3 Fuentes

- **Marca "DogApp":** fuente externa redondeada y muy bold, como *Baloo 2 ExtraBold*, *Fredoka One* o *Nunito ExtraBold*. Se descarga el `.ttf`, se copia en `res/font/dogapp_brand.ttf` y se usa con `android:fontFamily="@font/dogapp_brand"`. Cumple el criterio "no olvide usar fuentes externas que imiten ese estilo".
- **Títulos de pantalla y contenido:** la sans del sistema (`sans-serif-medium` / `sans-serif-black`) es aceptable; si se quiere uniformidad, *Poppins SemiBold* para títulos.
- **Tamaños de referencia:** marca del login 40–44 sp; subtítulo del Home 12 sp; título del header 20–22 sp; nombre de mascota en el ítem 16 sp bold; síntoma 13–14 sp; turno `#n` 12 sp bold.

6.4 Archivos de recursos a crear

```
res/
|-- values/colors.xml            todos los tokens de arriba
|-- values/dimens.xml            spacing_8, radius_card_16, avatar_52 ...
|-- values/strings.xml           textos + array de sintomas
|-- values/themes.xml            Theme.DogApp (NoActionBar)
|-- font/dogapp_brand.ttf        fuente externa de la marca
|-- drawable/
|   |-- bg_header_home.xml       fondo oscuro, esquina inf-izq curva
|   |-- bg_toolbar_curved.xml    fondo oscuro, esquinas inf. curvadas
|   |-- bg_button_pink.xml       (o usar MaterialButton con cornerRadius)
|   |-- ic_back_pink.xml         flecha de atr\'as tintada de rosa
|   |-- ic_save.xml  ic_edit.xml  ic_delete.xml  ic_add.xml
|   `-- ic_dog_head.webp         cabeza de perro, fondo transparente
|-- color/til_stroke.xml         selector morado(focus) / gris(reposo)
|-- assets/fingerprint.json      animaci\'on Lottie de la huella
`-- mipmap-*/ic_launcher.*       \'icono propio dogApp (adaptive icon)
```

7 Especificación pantalla por pantalla

Cada subsección traduce los criterios del enunciado a implementación concreta. La columna "dónde" es el archivo que el profesor pedirá abrir si algo no se ve bien.

7.1 HU 1.0 — Ventana Login

Acta: como aplicación, quiero presentar un sistema de logueo con biometría dactilar para mejorar la experiencia de ingreso. **Tipo:** Fragment de inicio, sin toolbar.

C	Qué exige	Cómo se implementa
1	Fondo gris oscuro, sin toolbar	`Theme.DogApp` con `windowNoTitle=true` y `windowActionBar=false`; raíz del layout con `android:background="@color/dogapp_dark"`. Confirmar que el fragment no infla `MaterialToolbar`
2	Cabeza de perro arriba a la derecha, fondo transparente, solo la cabeza, optimizada	`ImageView` de un `.webp` con alfa recortado a la cabeza; anclado `topToTop` + `endToEnd` con márgen 24 dp; `scaleType="fitCenter"` y `layout_constraintWidth_percent="0.55"` para que escale en pantallas distintas. Exportar en `drawable-xxhdpi` y `drawable-xxxhdpi` (o vectorizar)
3	"DogApp" blanco bold, debajo de la imagen, centrado, fuente externa	`TextView` con `@font/dogapp_brand`, `textColor="@color/white"`, `textStyle="bold"`, 40–44 sp; `topToBottomOf` la imagen
4	Imagen dinámica de huella abajo al centro	`LottieAnimationView` con `lottie_fileName="fingerprint.json"` desde `assets/`, `autoPlay="true"`, `repeatCount="infinite"`; 120–160 dp, anclada abajo con márgen inferior 64 dp
5	Al tocar la huella: ventana emergente con título, subtítulo y "Cancelar"	`BiometricPrompt`. Título `"Autenticación con Biometría"`, subtítulo `"Ingrese su huella digital"`, `setNegativeButtonText("Cancelar")`. El aspecto lo dibuja el sistema y *varía entre dispositivos*: el enunciado lo permite explícitamente
6	Huella correcta → HU 2.0; incorrecta → mensaje y no entra	En `onAuthenticationSucceeded`: `navigate(R.id.action_login_to_home)` con `popUpTo=login inclusive=true` (para que "atrás" no vuelva al login). En `onAuthenticationError`: `Snackbar` con el mensaje del sistema y *ninguna* navegación
7	Ícono propio de la app llamado dogApp	Adaptive icon: `mipmap-anydpi-v26/ic_launcher.xml` con `foreground` (cabeza de perro) + `background` de color; `ic_launcher_round`; `android:label="dogApp"` en el manifest

```
Manifest: <uses-permission android:name="android.permission.USE_BIOMETRIC"/>
Gradle:   implementation("androidx.biometric:biometric:1.1.0")

val prompt = BiometricPrompt(requireActivity(),
    ContextCompat.getMainExecutor(requireContext()),
    object : BiometricPrompt.AuthenticationCallback() {
        override fun onAuthenticationSucceeded(
            result: BiometricPrompt.AuthenticationResult) {
            findNavController().navigate(R.id.action_login_to_home)
        }
        override fun onAuthenticationError(code: Int, msg: CharSequence) {
            Snackbar.make(binding.root, msg, Snackbar.LENGTH_LONG).show()
        }
    })
```

```
val info = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Autenticacion con Biometria")
    .setSubtitle("Ingrese su huella digital")
    .setNegativeButtonText("Cancelar")
    .setAllowedAuthenticators(BiometricManager.Authenticators.BIOMETRIC_STRONG)
    .build()
prompt.authenticate(info)
```

Prueba de la huella en el emulador

Sin dedo enrolado, el diálogo no aparece y parece que "la biometría no funciona". En el emulador: *Settings* → *Security* → *Fingerprint* y enrolar un dedo; luego, con la app en el diálogo, mandar `adb -e emu finger touch 1` (o usar *Extended Controls* → *Fingerprint*). Verifica también el camino de error (huella no reconocida) y el botón Cancelar: los tres son criterios calificables.

7.2 HU 2.0 — Home Administrador de Citas

Acta: como administrador, quiero visualizar las citas creadas para gestionar su atención. **Tipo:** Fragment con RecyclerView + FAB.

C	Qué exige	Cómo se implementa
1	El botón atrás del teléfono debe llevar al escritorio, no al login	`requireActivity().onBackPressedDispatcher.addCallback(viewLifecycleOwner){ requireActivity().finishAffinity() }`. Además, el login se elimina del back stack al navegar (ver HU 1.0, criterio 6)
2	Fondo blanco; header gris oscuro con la esquina inferior izquierda curvada; título blanco bold "Administrador de Citas" a la izquierda y centrado vertical; a la derecha la imagen del perro con "DogApp" debajo	`LinearLayout`/`ConstraintLayout` con `background="@drawable/bg_header_home"` (shape sólido `#2E2E33` con `bottomLeftRadius=48dp` y `bottomRightRadius=0dp`). Texto en 2 líneas a la izquierda; a la derecha `ImageView` de la cabeza + `TextView` "DogApp" en 12 sp debajo
3	Ítems: fondo blanco, bordes redondeados, elevación; imagen circular de la mascota a la izquierda; nombre y síntoma al centro; turno arriba a la derecha	`item_cita.xml`: `MaterialCardView` (radio 16 dp, elevación 4 dp) con `ShapeableImageView` circular 52 dp, `tvNombre` bold 16 sp + `tvSintoma` 13 sp gris, y `tvTurno` con texto `"#"+id` en rosa bold arriba a la derecha
4	FAB rosado `#F05861` abajo a la derecha que lleva a Nueva Cita	`FloatingActionButton` con `app:backgroundTint="@color/dogapp_pink"`, `srcCompat="@drawable/ic_add"`, `layout_gravity="bottom|end"`; `setOnClickListener { navigate(R.id.action_home_to_nueva) }`
5	Al tocar un ítem, ir al Detalle de esa cita	`adapter.setOnItemClick { navigate(R.id.action_home_to_detalle, bundleOf("citaId" to it.id)) }` o Safe Args
6	La lista sale de Room (SQLite)	`viewModel.citas.observe(viewLifecycleOwner) { adapter.submitList(it) }` — `LiveData` generado por el DAO

Cómo pintar la esquina curvada del header sin pelear con XML

Dos caminos válidos: **(a)** un `shape` drawable con `<corners android:bottomLeftRadius="48dp" android:bottomRightRadius="0dp"/>` y `<solid android:color="@color/dogapp_dark"/>`; **(b)** un `MaterialCardView`/`ImageView` con `shapeAppearanceOverlay` usando `cornerFamily=rounded` y `cornerSizeBottomLeft=48dp`, `cornerSizeBottomRight=0dp`. La opción (b) es la misma que se reutiliza en las toolbars de las HU 3.0, 4.0 y 5.0, así que conviene resolverla una vez y reutilizar el estilo.

7.3 HU 3.0 — Ventana Nueva Cita

Acta: como administrador, quiero crear una cita para llevar el registro de los pacientes. **Tipo:** Fragment de formulario, con toolbar personalizada.

C	Qué exige	Cómo se implementa
1	Toolbar oscura con esquinas inferiores curvadas, título "Nueva Cita" centrado blanco bold, flecha de atrás rosada que vuelve al Home	`MaterialToolbar` con `background="@drawable/bg_toolbar_curved"`, `app:navigationIcon="@drawable/ic_back_pink"` y `app:title` centrado (`app:titleCentered="true"` o `TextView` centrado). `setNavigationOnClickListener { findNavController().navigateUp() }`
2	Campo "Nombre de la Mascota": hint gris, flota al enfocar, borde morado al enfocar, máximo 15 caracteres	`TextInputLayout` estilo `Widget.MaterialComponents.TextInputLayout.OutlinedBox` + `TextInputEditText`, `hint="@string/nombre_mascota"`, `android:maxLength="15"`, `app:boxStrokeColor="@color/til_stroke"`. El flotado del hint es nativo del `TextInputLayout`
3	Campo "Raza" con **autocompletado**: al escribir 2 letras aparecen las razas que *comienzan* por ellas, alimentado por la API con Retrofit	`MaterialAutoCompleteTextView` dentro de `TextInputLayout` (`endIconMode="dropdown_menu"`), `android:completionThreshold="2"`. El adapter se llena con `viewModel.razas.observe` y su `Filter` usa `startsWith(prefix, ignoreCase=true)`
4	Campo "Nombre del Propietario", máximo 30	Igual que el criterio 2, `android:maxLength="30"`
5	Campo "Teléfono", solo números, máximo 10	`android:inputType="number"`, `android:maxLength="10"`, `digits="0123456789"`
6	Lista de síntomas con el ítem "Síntomas" por defecto (quemada en la app)	`MaterialAutoCompleteTextView` no editable (`inputType="none"`, `endIconMode="dropdown_menu"`) con `ArrayAdapter` de `R.array.sintomas`; el índice 0 es "Síntomas" = sin selección. Los seis valores: Solo duerme, No come, Fractura extremidad, Tiene pulgas, Tiene garrapatas, Bota demasiado pelo
7	Botón rosado, redondeado, con ícono a la izquierda, texto "Guardar Cita"; **inactivo** mientras falte nombre, raza, propietario o teléfono	`MaterialButton` con `app:cornerRadius="24dp"`, `backgroundTint="@color/dogapp_pink"`, `app:icon="@drawable/ic_save"`, `iconGravity="textStart"`. Estado en `doAfterTextChanged`: `btn.isEnabled = nombre.isNotBlank() && raza.isNotBlank() && propietario.isNotBlank() && telefono.isNotBlank()`

C	Qué exige	Cómo se implementa
8	Si todo está lleno pero no hay síntoma: al pulsar, mensaje “Selecciona un síntoma”	En el `onClick`: `if (sintomaSeleccionado == null) { Snackbar/AlertDialog("Selecciona un síntoma"); return }`
9	El botón se habilita cuando los cuatro campos están llenos y el texto pasa a blanco bold	Además de `isEnabled`, cambiar `setTextColor(Color.WHITE)` + `Typeface.BOLD` al habilitar, y `alpha=0.5` + color rosado claro mientras esté inactivo
10	Al guardar: insertar en Room y navegar al Home, donde debe verse la cita	`viewModel.guardar(cita)` en `viewModelScope`; al terminar (`onSuccess`), `navigate(R.id.action_nueva_to_home)` con `popUpTo` para no dejar el formulario en la pila

El filtro del autocompletado: el error más común del proyecto

El `ArrayAdapter` por defecto filtra por “contiene” y respeta el orden de la lista, así que escribir “pu” puede devolver “bullpug” antes que “pug”. Para cumplir “que comiencen por esas letras” hay que sobrescribir el `Filter` con la lógica que sigue.

```
override fun getFilter(): Filter = object : Filter() {
    override fun performFiltering(prefix: CharSequence?): FilterResults {
        val p = prefix?.toString()?.trim()?.lowercase().orEmpty()
        val out = if (p.length < 2) emptyList()
                  else razas.filter { it.lowercase().startsWith(p) }
        return FilterResults().apply { values = out; count = out.size }
    }
    override fun publishResults(c: CharSequence?, r: FilterResults?) {
        clear()
        addAll(r?.values as? List<String> ?: emptyList())
        notifyDataSetChanged()
    }
}
```

Y no olvides que `performFiltering` corre en hilo de fondo: no llames a la API ahí. Pide la lista una sola vez y filtra en memoria.

7.4 HU 4.0 — Ventana Detalle de la Cita

Acta: como administrador, quiero ver el detalle de un paciente para decidir. **Tipo:** Fragment de solo lectura + dos FAB.

C	Qué exige	Cómo se implementa
1	Toolbar oscura curvada; el título es el **nombre de la mascota**; flecha rosada que vuelve al Home	Reutilizar el estilo de la HU 3.0; `toolbar.title = cita.nombreMascota` dentro del `observe`. Como el título depende de datos, hay que asignarlo después de cargar el `LiveData`
2	Card con la foto de la mascota, con margen, **por encima** de otra tarjeta blanca	`MaterialCardView` (radio 16 dp, elevación 8 dp) con `ShapeableImageView` (`scaleType="centerCrop"`) y `layout_margin="20dp"`. Para superponer, `ConstraintLayout` con la segunda card anclada debajo y un `layout_marginTop="-32dp"` en la card de información
3	Segunda card: turno arriba a la derecha, raza en negro bold, síntoma, línea delgada rosada, "Propietario" + nombre, "Teléfono" + número	Orden vertical: `tvTurno` (`"#"+id`, rosa, bold, `end|top`), `tvRaza` (negro bold, centrado), `tvSintoma`, `View` de 1 dp con `background="@color/dogapp_pink"` y ancho 90 %, `tvPropietario`, `tvTelefono`
4	FAB rosado con ícono de eliminación, abajo a la **izquierda**: borra en Room y vuelve al Home sin la cita	`FloatingActionButton layout_gravity="bottom|start"`, `backgroundTint="@color/dogapp_pink"`, `ic_delete`. `viewModel.eliminar(cita); findNavController().navigate(R.id.action_detalle_to_home)`. No se exige confirmación
5	FAB azul oscuro con ícono de edición, abajo a la **derecha**: lleva a Editar Cita	`FloatingActionButton layout_gravity="bottom|end"`, `backgroundTint="@color/dogapp_edit_blue"`, `ic_edit`; navega pasando el mismo `citaId`

```
// el detalle se alimenta del id que llega por navegacion
private val citaId: Int by lazy { requireArguments().getInt("citaId") }
viewModel.cargar(citaId)
viewModel.cita.observe(viewLifecycleOwner) { cita ->
    cita ?: run { findNavController().navigateUp(); return@observe }
    toolbar.title = cita.nombreMascota
    tvRaza.text = cita.raza
    tvSintoma.text = cita.sintoma
    tvTurno.text = "#${cita.id}"
    tvPropietario.text = "Propietario: ${cita.propietario}"
    tvTelefono.text = "Telefono: ${cita.telefono}"
    Glide.with(this).load(cita.imagenUrl).circleCrop().into(imgMascota)
}
```

La superposición de tarjetas es el criterio más fácil de perder

El texto dice "este contenedor está por encima de otra tarjeta de color blanco". Dos `MaterialCardView` apiladas sin más no lo logran: la de la imagen debe tener mayor `elevation` **y** solaparse físicamente con la de abajo (margen superior negativo, o anclando la de arriba con `layout_marginBottom="-32dp"`). Verifícalo en pantalla: si se ven dos rectángulos separados con un hueco, el criterio no está cumplido.

7.5 HU 5.0 — Ventana Editar Cita

Acta: como administrador, quiero modificar datos de la mascota para mantener actualizada la agenda.
Tipo: mismo formulario de la HU 3.0, precargado.

C	Qué exige	Cómo se implementa
1	Toolbar "Editar Cita", flecha rosada que vuelve al **Detalle** (mostrando la mascota editada)	`navigateUp()` (no navegar al Home). El detalle se refresca solo porque observa `LiveData` de Room
2	Nombre de la mascota, máximo 15, **precargado** desde el detalle	`TextInputEditText.setText( cita.nombreMascota)` en el `observe`; no volver a pedir el id al usuario: llega por argumento de navegación
3	Raza con el mismo autocompletado por API (2 letras, prefijo)	Reutilizar el adapter de la HU 3.0 (mismo código, mismo ViewModel de formulario)
4	Nombre del propietario, máximo 30, precargado	`setText(cita.propietario)`
5	Teléfono, solo números, máximo 10, precargado	`setText(cita.telefono)`
6	Botón rosado redondeado con ícono y texto "Editar Cita"; inactivo si falta algún campo	Misma lógica de habilitación que la HU 3.0, con `app:icon="@drawable/ic_edit"`
7	Al habilitarse, el texto pasa a blanco bold	Igual que la HU 3.0, criterio 9
8	Al guardar: `update` en Room y navegar al Home con la cita modificada	`viewModel.actualizar(cita)` y `navigate( R.id.action_editar_to_home)`. Si cambiaste la raza, **regenera la foto** con `fotoDeRaza(nuevaRaza)` antes del `update` (ver el "Importante" del enunciado)

Lo que el "Importante" de la HU 5.0 obliga a hacer

"Si al momento de editar la cita la raza escogida fue Akita, la imagen debe mostrar un perro alusivo a esa raza". Es decir: cambiar la raza **tiene** que cambiar la foto del ítem en el Home. Si guardas la misma `imagenUrl` vieja, ese requisito queda incumplido y se nota en la demo (el profesor edita la raza y compara la foto). Regla: *cada guardado con raza distinta pide una URL nueva a la API.*

8 Navegación: un solo Activity, cinco Fragments

El item "Uso de Fragment" y el item "Uso de Navigation" se califican juntos y son de los más rápidos de verificar: si las pantallas son Activities, o si se navega con `Intent` y `startActivity`, se pierden dos items del 60 %.

Desde	Hacia	Acción	Detalle
`loginFragment`	`homeFragment`	`action_login_to_home`	Solo después de biometría exitosa; `popUpTo= loginFragment, popUpToInclusive=true`

Desde	Hacia	Acción	Detalle
`homeFragment`	`nuevaCitaFragment`	`action_home_to_nueva`	FAB rosado
`homeFragment`	`detalleFragment`	`action_home_to_detalle`	`argument citaId`, tipo `integer`
`nuevaCitaFragment`	`homeFragment`	`action_nueva_to_home`	`popUpTo=home` (no deja el formulario en la pila)
`detalleFragment`	`homeFragment`	`action_detalle_to_home`	Al eliminar, o con la flecha de atrás
`detalleFragment`	`editarFragment`	`action_detalle_to_editar`	`argument citaId`
`editarFragment`	`homeFragment`	`action_editar_to_home`	Tras `update` exitoso
`editarFragment`	`detalleFragment`	(botón atrás)	`navigateUp()` al detalle

```
<fragment android:id="@+id/loginFragment"
    android:name="com.univalle.dogapp.view.login.LoginFragment"
    android:label="Login">
    <action android:id="@+id/action_login_to_home"
        app:destination="@id/homeFragment"
        app:popUpTo="@id/loginFragment" app:popUpToInclusive="true"/>
</fragment>

<fragment android:id="@+id/detalleFragment"
    android:name="com.univalle.dogapp.view.detalle.DetalleFragment">
    <argument android:name="citaId" app:argType="integer"/>
    <action android:id="@+id/action_detalle_to_editar"
        app:destination="@id/editarFragment"/>
</fragment>
```

- `MainActivity` contiene `NavHostFragment` con `app:defaultNavHost="true"` y `app:navGraph="@navigation/nav_graph"`.
- `startDestination` = `loginFragment`.
- Las flechas de las toolbars llaman a `navigateUp()` o a la acción explícita; nunca `startActivity`.
- Opcional pero recomendable: Safe Args (`androidx.navigation.safeargs.kotlin`) para no escribir el nombre del argumento como texto.

9 Jira y GitHub: el 20 % que suele perderse por descuido

Es frecuente que un equipo tenga la app funcionando y aun así saque 4,0 porque no puede demostrar gestión. El RA-3 vale tanto como la interfaz completa, y se construye *durante* el sprint, no la noche anterior.

9.1 Jira: cómo reflejar los criterios

Elemento	Qué crear
Proyecto	Tipo Scrum, nombre `DogApp` o el nombre del equipo
Épica	`DogApp - Sprint 1`. Agrupa las 5 historias
Historias	5 issues tipo *Historia*: `HU 1.0 Ventana Login`, `HU 2.0 Home`, `HU 3.0 Nueva Cita`, `HU 4.0 Detalle`, `HU 5.0 Editar`. En la descripción, el acta completa y los criterios de aceptación
Criterios	Los criterios del enunciado como *criterios de aceptación* (checklist) o como subtareas: es lo que el docente revisa
Subtareas	Al menos: `UI/XML`, `ViewModel + LiveData`, `Room`, `Retrofit`, `Navegación`, `QA`. Asignadas a personas distintas
Tablero	Columnas **Por hacer**, **En curso**, **Hecho**, con WIP razonable y tarjetas moviéndose a diario
Responsables	Cada historia y subtarea con asignado real (es la evidencia para la nota individual)
Evidencia	Capturas del tablero en el PDF de entrega y en el README

La prueba que hace el docente

Abre Jira y mira el *historial de actividad*. Si las 5 historias se movieron el mismo día, o si todo lo hizo una sola cuenta, la gestión queda en evidencia como simulada. Mueve las tarjetas el mismo día que trabajas y comenta el avance en la historia ("se implementó el autocompletado, falta el filtro por prefijo"). Eso es exactamente lo que dice la rúbrica: "que le permita construir aplicaciones de manera colaborativa".

9.2 GitHub: reglas del repositorio

- **Nombre:** igual al nombre del equipo (`equipoUno`, `equipoDos`, ...). El enunciado lo exige dos veces (repositorio y archivo PDF).
- **Ramas:** `main` (lo entregable) + `develop` + `feature/hu1-login`, `feature/hu2-home`, `feature/hu3-nueva`, `feature/hu4-detalle`, `feature/hu5-editar`. Una rama por historia y Pull Request hacia `develop`.
- **Commits:** convencionales y pequeños (`feat(login): biometria con BiometricPrompt`, `fix(home): refrescar lista al eliminar`). Que cada integrante commitee con su **correo de GitHub** configurado: los commits sin autor identificable no cuentan como evidencia individual.
- **Pull Requests:** uno por HU, con al menos una revisión de otro integrante (`Approve`). Esa interacción es la "evidencia del trabajo en equipo".
- `.gitignore`: `.idea/`, `build/`, `local.properties`, `*.iml`, `.gradle/`. **No** subir `local.properties` (contiene la ruta del SDK).
- **README.md**: integrantes, qué hace la app, cómo compilarla, capturas de las 5 pantallas, diagrama de paquetes MVVM, endpoints usados, decisiones técnicas (incluido el tema del endpoint de la foto por raza) y enlaces a Jira.
- **Tag** `v1.0-sprint1` en el commit de entrega.
- **Congelar el repositorio** antes del 25 de noviembre, 6:30 p.m.: cualquier commit posterior cuesta 0,2.

10 Plan de ejecución del sprint

El sprint va del 11 al 25 de noviembre de 2026, con monitoría el 18. Este orden es el que menos retrabajo genera: primero el esqueleto que se compila y navega, después los datos, después la red y al final el pulido visual.

Fase	Fechas	Entregable verificable	Riesgo
0. Arranque	11–12 nov	Repo `equipoX` creado, Jira con épica + 5 historias + subtareas, `MainActivity` + `NavHostFragment` + 5 Fragments vacíos que navegan, temas y colores en `values`	Bajo
1. Login + marca	13–14 nov	HU 1.0 completa: fondo, cabeza de perro, fuente, Lottie, biometría, navegación con `popUpTo`, ícono de app	Medio (biometría en emulador)
2. Room + Home	15–17 nov	Entidad, DAO, DB, Repository, ViewModel; Home con RecyclerView leyendo de Room; alta manual de prueba; **back del Home cierra la app**	Medio
3. Formularios	15–18 nov	HU 3.0 y HU 5.0 con validaciones, máximos, `TextInputLayout`, botón que se habilita, mensaje “Selecciona un síntoma”, insertar y actualizar	Alto (estados del botón)
4. Monitoría	18 nov	Demo parcial: login → home → nueva → home. Anotar correcciones del docente	—
5. Retrofit	19–20 nov	Lista de razas, autocompletado por prefijo con umbral 2, foto por raza con Glide, fallback sin red	Alto (filtro)
6. Detalle	21 nov	HU 4.0 con tarjetas superpuestas, turno, divisor rosado, eliminar y editar	Medio
7. Integración	22–23 nov	Recorrido completo de las 5 HU, regeneración de foto al editar raza, tolerancia a fallos de red	Alto
8. QA + pulido	23–24 nov	Checklist de la sección 12 al 100 %, README, capturas, PDF de entrega	Medio
9. Entrega	25 nov	PDF `equipoX.pdf` subido por un solo integrante, tag, repositorio congelado antes de las 6:30 p.m.	—

Reparto sugerido para que todos tengan nota individual

Con 4 integrantes: **A** arquitectura + Room + Repository + ViewModel del Home; **B** Login + biometría + sistema de diseño (colores, fuentes, formas, ícono); **C** Retrofit + autocompletado + Glide + fallback; **D** Detalle + Editar + QA y Jira. Todos commitean, todos mueven tarjetas y todos pueden explicar la capa que hicieron. Si son 3 o 5, se reparte igual: nadie queda sin capa propia.

11 Errores que cuestan nota (y su antídoto)

Error	Antídoto
Cinco Activities en lugar de cinco Fragments: se pierden los items de Fragment y Navigation	`MainActivity` única + `NavHostFragment` + `nav_graph`. Antes de escribir la primera pantalla, verificar que navega
Toda la lógica en el Fragment: se pierde el item MVVM	ViewModel por pantalla; el Fragment solo observa y pinta
DAO que devuelve `List` en vez de `LiveData`: la lista no se refresca al guardar o borrar	`LiveData<List<CitaEntity>>` en `observarTodas()`
Llamar a la API en cada tecla del autocompletado	Una carga de razas en `init`, filtrado en memoria
Mostrar una foto aleatoria aunque se eligió una raza	`/breed/{raza}/images/random` y regenerar al editar la raza
Guardar el teléfono como número	Guardar como `String` con `inputType="number"`
El botón Guardar arranca habilitado o nunca se habilita	Estado calculado con `doAfterTextChanged` sobre los cuatro campos + `isEnabled`
El botón habilitado no cambia el texto a blanco bold	`setTextColor` + `Typeface.BOLD` al habilitar (criterio 9 de HU 3.0 y criterio 7 de HU 5.0)
No hay mensaje “Selecciona un síntoma”	Validar el índice 0 del dropdown antes de guardar
El back del Home devuelve al Login	`finishAffinity()` en el dispatcher + `popUpTo` inclusive al entrar al Home
Tarjetas del detalle separadas en vez de superpuestas	Margen negativo o anclaje con solapamiento; comprobar en pantalla
Toolbars con título alineado a la izquierda	Título centrado según los pantallazos (Nueva Cita y Detalle)
Se olvida el ícono propio de la app	Adaptive icon `ic_launcher` + `android:label="dogApp"`
La app se cae sin internet	`runCatching` + imagen de marcador; el CRUD local debe funcionar offline
Falta la animación Lottie porque el JSON no está en `assets`	Copiar el `.json` a `app/src/main/assets/` y usar `lottie_fileName`
Commits todos de la misma persona	Reparto por capas + verificar `git shortlog -sne` antes de entregar

12 Checklist de aceptación

Marcar cada casilla sólo después de verlo en el dispositivo. Es la lista que se usa como guion de la sustentación: cada línea es un criterio calificable.

12.1 Arquitectura (RA-1, 60 %)

	Verificación	Hecho
1	Existen los paquetes `model`, `view`, `viewmodel`, `repository`, `data`	☐
2	Ningún Fragment importa `CitaDao` ni `DogApiService`	☐
3	Ningún ViewModel recibe `Context`	☐
4	El Repository centraliza Room y Retrofit	☐
5	La lista del Home se actualiza sola al insertar, editar y borrar	☐
6	Los datos sobreviven al cerrar y reabrir la app (persistencia real)	☐
7	Room con `@Entity`, `@Dao`, `@Database`	☐
8	Retrofit con `@GET`, `@Path`, Gson y `suspend`	☐
9	Las 5 pantallas son Fragments dentro de un `nav_graph`	☐
10	`citaId` viaja por argumento de navegación	☐
11	Corrutinas en todas las operaciones de datos	☐

12.2 Interfaz (RA-2, 20 %)

	Verificación	Hecho
12	Login sin toolbar, fondo `#2E2E33`, cabeza de perro con transparencia arriba a la derecha	☐
13	“DogApp” blanco bold, centrado, con fuente externa	☐
14	Huella Lottie animada abajo al centro	☐
15	Diálogo biométrico con título, subtítulo y Cancelar	☐
16	Home con header oscuro, esquina inferior izquierda curvada, título y logo	☐
17	Ítems blancos redondeados con elevación, avatar circular, nombre, síntoma y turno `#n`	☐
18	FAB rosado de crear abajo a la derecha	☐
19	Toolbars oscuras con esquinas inferiores curvadas y flecha rosada	☐
20	Campos con hint que flota, borde morado con foco, máximos 15/30/10	☐
21	Lista de síntomas con “Síntomas” por defecto	☐
22	Botón rosado redondeado con ícono, inactivo hasta completar, blanco bold al habilitarse	☐
23	Detalle con foto superpuesta sobre la tarjeta de datos	☐
24	Detalle con turno, raza bold, síntoma, línea rosada, propietario y teléfono	☐
25	FAB de eliminar rosado a la izquierda y de editar azul oscuro a la derecha	☐
26	Formulario de edición precargado con los datos del detalle	☐
27	Ícono de la app propio (`dogApp`)	☐

12.3 Funcionamiento y gestión (RA-1 funcional + RA-3)

	Verificación	Hecho
28	Huella correcta entra al Home; huella fallida muestra error y no entra	☐
29	Cancelar sale del diálogo y permanece en el login	☐
30	El back del Home lleva al escritorio, no al login	☐
31	Guardar sin síntoma muestra “Selecciona un síntoma” y no guarda	☐
32	Guardar navega al Home y la cita aparece con su turno y su foto de raza	☐
33	Editar cambia los datos y la foto si cambia la raza	☐
34	Eliminar borra de Room y la cita desaparece del Home	☐
35	Con el móvil en modo avión la app no se cae y permite crear citas	☐
36	Jira: 5 historias con criterios, subtareas, responsables y movimientos durante el sprint	☐
37	GitHub: repo con el nombre del equipo, ramas por HU, PRs revisados, commits de todos	☐
38	README con integrantes, capturas, arquitectura y enlaces	☐
39	PDF `equipoX.pdf` con nombres, apellidos, enlace al repo y a Jira	☐
40	Repositorio congelado antes de las 6:30 p.m. del 25 de noviembre	☐

13 Entrega y sustentación

Requisito	Detalle del enunciado
Cámara	Todos los integrantes con cámara encendida durante la sustentación. Penalización de 0,5
Originalidad	Cualquier indicio de copia se penaliza según los estatutos de la Universidad
Formato	Presentación en equipo, no individual
Puntualidad	Hora de inicio: 6:30 p.m.
Herramientas	Cada integrante con repositorio, Jira, Android Studio abierto y dispositivo o emulador listo para mostrar la app
Subida del PDF	Un solo integrante sube el PDF; el archivo se llama como el equipo (`equipoUno.pdf`)
Contenido del PDF	Nombres y apellidos de los integrantes + enlace al repositorio + enlace a Jira
Repositorio	Se llama como el equipo (`equipoUno`)
Commits	Ningún commit después de la fecha y hora de entrega. Penalización de 0,2

13.1 Guion sugerido de sustentación (10–12 minutos)

1. **(30 s)** Abrir Jira y mostrar el tablero con las columnas y las 5 historias; abrir GitHub y mostrar la pestaña de commits y PRs.
2. **(2 min)** Login: marca, Lottie, toque, diálogo biométrico, huella correcta → Home. Repetir mostrando *Cancelar* y una huella fallida.

3. **(1 min)** Home: header, ítems con foto de la raza y turno. Pulsar atrás del teléfono y mostrar que sale al escritorio.
4. **(2 min)** Nueva Cita: mostrar el botón inactivo, llenar los cuatro campos, mostrar el autocompletado con dos letras, dejar el síntoma vacío y provocar "Selecciona un síntoma", luego guardar y ver la cita en el Home.
5. **(1,5 min)** Detalle: tarjetas superpuestas y todos los datos. Editar: cambiar la raza, guardar y volver a mostrar el Home con la foto nueva.
6. **(1 min)** Detalle → eliminar → Home sin la cita.
7. **(2 min)** Código: abrir el árbol de paquetes y mostrar Fragment → ViewModel → Repository → DAO/Retrofit. Mostrar `observarTodas()` devolviendo `LiveData` y la interfaz de Retrofit.
8. **(1 min)** Cierre: decisiones técnicas (endpoint de foto por raza, manejo de red caída, LiveData como fuente única) y reparto de trabajo.

Antes de encender la câmara

Compilar en *Release* o al menos en *Debug* limpio, con el emulador ya encendido y la huella enrolada, la app instalada en el dispositivo y el repositorio descargado en el portátil de cada integrante. La sustentación tiene un tiempo fijo: cada minuto gastado en resolver un `Gradle sync` fallido es un criterio que no se alcanza a mostrar.

14 Anexo A: dependencias de `build.gradle.kts`

Todas estas líneas se añaden al módulo `app`, partiendo de la configuración de la clase 8 (`compileSdk = 34`, `minSdk = 24`, KSP para Room).

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("com.google.devtools.ksp")
    id("androidx.navigation.safeargs.kotlin")   // opcional pero recomendado
}

android {
    namespace = "com.univalle.dogapp"
    compileSdk = 34
    buildFeatures { viewBinding = true }        // o dataBinding = true
}

dependencies {
    // base
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.11.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    // MVVM
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")
    implementation("androidx.activity:activity-ktx:1.8.2")
    implementation("androidx.fragment:fragment-ktx:1.6.2")

    // Navigation
    implementation("androidx.navigation:navigation-fragment-ktx:2.7.6")
    implementation("androidx.navigation:navigation-ui-ktx:2.7.6")

    // Room
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")

    // Retrofit
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

    // Imagenes y animacion
```

```
    implementation("com.github.bumptech.glide:glide:4.16.0")
    implementation("com.airbnb.android:lottie:6.3.0")

    // Biometria
    implementation("androidx.biometric:biometric:1.1.0")

    // RecyclerView y CardView
    implementation("androidx.recyclerview:recyclerview:1.3.2")
    implementation("androidx.cardview:cardview:1.0.0")

    // Corrutinas
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
}
```

15 Anexo B: código mínimo de las tres piezas delicadas

B.1 Back del Home al escritorio

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    requireActivity().onBackPressedDispatcher.addCallback(viewLifecycleOwner) {
        requireActivity().finishAffinity()   // cierra la tarea -> escritorio
    }
}
```

B.2 Boton que se habilita solo y cambia a blanco bold

```
private fun vigilarCampos() {
    val revisar = {
        val listo = etNombre.text.isNullOrBlank().not() &&
                    etRaza.text.isNullOrBlank().not() &&
                    etPropietario.text.isNullOrBlank().not() &&
                    etTelefono.text.isNullOrBlank().not()
        btnGuardar.isEnabled = listo
        btnGuardar.setTextColor(
            if (listo) Color.WHITE else Color.parseColor("#F5A9AD"))
        btnGuardar.setTypeface(null, if (listo) Typeface.BOLD else Typeface.NORMAL)
        btnGuardar.alpha = if (listo) 1f else 0.6f
    }
    listOf(etNombre, etRaza, etPropietario, etTelefono).forEach { campo ->
        campo.doAfterTextChanged { revisar() }
    }
    revisar()
}
```

B.3 Guardar con foto por raza y navegar

```
btnGuardar.setOnClickListener {
```

```
    val sintoma = dropdownSintomas.text.toString()
    if (sintoma == "Sintomas" || sintoma.isBlank()) {
        Snackbar.make(binding.root, "Selecciona un sintoma",
            Snackbar.LENGTH_LONG).show()
        return@setOnClickListener
    }
    viewModel.guardar(
        nombre = etNombre.text.toString().trim(),
        raza   = etRaza.text.toString().trim(),
        dueno  = etPropietario.text.toString().trim(),
        tel    = etTelefono.text.toString().trim(),
        sintoma = sintoma
    )
}

// en el ViewModel
fun guardar(nombre: String, raza: String, dueno: String,
            tel: String, sintoma: String) {
    viewModelScope.launch {
        val url = repository.fotoDeRaza(raza)          // API, con fallback
        repository.guardar(CitaEntity(
            nombreMascota = nombre, raza = raza, propietario = dueno,
            telefono = tel, sintoma = sintoma, imagenUrl = url,
            fechaCreacion = System.currentTimeMillis()
        ))
        _guardado.emit(true)                           // el Fragment navega
    }
}
```

Nota final de uso de este documento

Este documento no reemplaza el enunciado: lo descompone. Si hay conflicto entre un valor de color, un radio o un texto y lo que dice el PDF oficial de las Historias de Usuario, **prevalece el PDF oficial**. Los valores de color y medidas aquí son lecturas razonables de los pantallazos, pensadas para centralizarse en `res/values` y ajustarse una sola vez.